Software Requirements Specification for Software Engineering: CAS Course Deadline Management System

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

October 20, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 1
- 1.2 Scope . . . 1
- 1.3 Stakeholders . . . 2
- 1.4 Definitions and Acronyms . . . 2
- 1.5 Characteristics of Intended Reader . . . 4

2 System Overview **4**
- 2.1 Problem Statement . . . 5
- 2.2 Product Vision . . . 5
- 2.3 System Context . . . 5
- 2.4 User Characteristics . . . 7
- 2.5 System Constraints . . . 7

3 Stakeholder Requirements **8**

4 User Personas **9**
- 4.1 Persona 1 . . . 9
- 4.2 Persona 2 . . . 10
- 4.3 Persona 3 . . . 10

5 Use Cases **11**
- 5.1 UC1: Submit Course Outline . . . 11
- 5.2 UC2: Review Extracted Course Information . . . 12
- 5.3 UC3: Review Course Outline Against McMaster and CAS Policies . . . 13
- 5.4 UC4: Review Course Outline Against CEAB Criteria . . . 14
- 5.5 UC5: Review Course Outline Findings . . . 14
- 5.6 UC6: Submit Tentative Assessment Dates . . . 15
- 5.7 UC7: Analyze Assessment Schedule . . . 16
- 5.8 UC8: Generate Scheduling Recommendations . . . 17
- 5.9 UC9: Review and Decide on Scheduling Recommendations . . . 18
- 5.10 UC10: Update Assessment Dates and Re-analyze Schedule . . . 18

6 Functional Requirements **19**

7 Data Requirements **21**
- 7.1 Major Data Entities . . . 21
- 7.2 Data Dictionary . . . 22
- 7.3 Privacy and Data Retention . . . 23

8 Non-Functional Requirements **24**

9 AI and Machine Learning Requirements **26**
9.1 Model Inputs . . . 27
9.2 Model Outputs . . . 27
9.3 Evaluation Metrics . . . 27
9.4 Known Limitations . . . 28

10 Requirements Traceability Matrices **29**

11 Rationale **31**
11.1 Assumptions . . . 32
11.2 System Constraints . . . 33
11.3 Functional Requirements . . . 33
11.4 Nonfunctional Requirements . . . 35
11.5 Data Requirements . . . 35
11.6 AI and Machine Learning . . . 36
11.7 Numeric Values . . . 36

12 Likely Changes **36**

13 Unlikely Changes **38**

14 Appendices **39**
14.1 Appendix C: Values of Auxiliary Constants . . . 39
14.2 Appendix D: Generative AI Disclosure . . . 41
14.3 AI Tools Used . . . 41
14.4 How and Where Used . . . 41
14.5 Verification of AI-Generated Content . . . 41
14.6 Appendix E: Reflection . . . 42

Revision History

Date	Version	Developer(s)	Notes
October 19, 2026	1.0	S. Mehta, A. Bornomala, H.S. Sagar, V. Lai	Initial version

1 Introduction

1.1 Purpose

The CAS (Computing and Software) Department currently manages course outlines and assessment schedules on a course-by-course basis. Course instructors set their deadlines independently, so students in the same program and year can face busy weeks of midterms and deadlines with no department-level visibility into them. CAS administrative staff must also manually check each outline against McMaster course outline policy [**?**] and the Canadian Engineering Accreditation Board (CEAB) requirements [**?**], and follow up with instructors when information is missing.

The CAS Deadline Management System is intended to make assessment dates visible across courses and to identify policy and accreditation gaps early. It reduces the manual effort of checking outlines against these requirements and helps avoid weeks with heavily clustered assessments. Instructors and administrators remain in the loop and responsible for every decision. The system only flags issues and makes suggestions, and never changes a course outline or assessment date on its own.

1.2 Scope

The CAS Deadline Management System aims to improve course outline review and assessment scheduling within the CAS Department at McMaster University. The system addresses the lack of department-wide visibility into assessment dates and the manual effort required to verify course outlines against university policies and CEAB requirements. The system will allow CAS administrators to upload course outlines, extract and store relevant course and assessment information, identify missing information and potential policy or accreditation gaps, and visualize assessment schedules across courses. It will also identify potential assessment conflicts and suggest alternative dates to help distribute assessments more evenly throughout the academic term. The system will support reviewing and exporting extracted information to facilitate administrative coordination.

The primary users are CAS administrators and course instructors, while students may benefit indirectly from improved assessment scheduling. The system is intended to reduce administrative workload, improve policy compliance, and support more balanced assessment schedules.

The system will provide recommendations for review rather than make decisions independently. It will not automatically modify course outlines, change assessment dates, or override instructor decisions. Instructors retain authority over their course schedules, and administrators remain responsible for reviewing policy compliance and accreditation requirements. The system's recommendations will be based on the available course outline data, timetable information, and defined policies; it will not guarantee that all scheduling conflicts or com-

pliance issues are identified.

1.3 Stakeholders

The CAS Midterm and Assignment Optimization System is intended to support the Computing and Software (CAS) department at McMaster University in reviewing course outlines provided by instructors and coordinating assessment schedules. The stakeholders include individuals who directly use the system, individuals affected by its recommendations, and organizations responsible for ensuring policy requirements and accreditation.

Primary Stakeholders:

- CAS Administration Team (Customer)
- CAS Department Professors and Instructors (End Users)
- Capstone Project Team 19 (Development Team)

Secondary Stakeholders:

- Software Engineering Students (Indirectly affected by assessment scheduling recommendations)
- McMaster University (Affected by the system's support for course coordination and policy compliance)

Tertiary Stakeholders:

- Canadian Engineering Accreditation Board (CEAB)
- Future CAS Administrators and Instructors

1.4 Definitions and Acronyms

Term	**Definition**
A	Assumption. Assumptions are numbered A1, A2, ... and listed in Section 11.1.
Academic calendar	The university's published dates for a term, including the start and end of classes, recesses, and the final examination period.
AIR	AI Requirement. AI and machine learning requirements are numbered AIR1, AIR2, ...

Term	Definition
AODA	Accessibility for Ontarians with Disabilities Act, the Ontario law that sets accessibility standards, including for websites [**?** **?**].
Assessment	Any graded component of a course, such as a midterm, assignment, quiz, lab, or project milestone.
CAS	Department of Computing and Software, McMaster University.
CEAB	Canadian Engineering Accreditation Board, the organization responsible for accrediting undergraduate engineering programs in Canada.
Cohort	The set of CAS courses that share a program and year level (Assumption A1). Used to estimate which assessments the same students face together.
Course Code	The identifier of a course, made up of a subject prefix and a course number (for example, SFWRENG 2DA4). The prefix identifies the program and the first digit of the number identifies the year level.
Course Outline	The official document for a course offering that lists its learning objectives, assessments, weights, and due dates, as required by the Undergraduate Course Management Policy [**?**].
CP-SAT	The constraint-programming solver in Google OR-Tools, used to compute suggested assessment dates [**?**].
Cross-Listed Course	A single course offered under more than one course code (for example, SFWRENG 4X03, COMPSCI 3X03 and MECHTRON 3X03).
DR	Data Requirement. Privacy and data retention requirements are numbered DR1, DR2, ...
FR	Functional Requirement.
GA	Graduate Attribute. One of the twelve outcomes the CEAB requires accredited engineering programs to develop in their graduates [**?**].
LC	Likely Change.
LLM	Large Language Model, an AI model trained on large amounts of text to understand and generate human language.
Master Course Sheet	The department's record of which graduate attributes each course is expected to cover.
MSAF	McMaster Student Absence Form, used by students to report short absences that affect academic work.
NFR	Nonfunctional Requirement.
PDF	Portable Document Format.
Restricted Period	A date range in which an assessment may not be due under university policy, such as a recess or the test and examination ban period.
RTM	Requirements Traceability Matrix.

Term	Definition
Scheduling Suggestion	An alternative date for an assessment, proposed by the system with an explanation. Suggestions are not binding; only the instructor can change a date.
SR	Stakeholder Requirement.
SRS	Software Requirements Specification, a document that describes a software system's requirements, including its expected functionality and constraints.
TBD	To Be Determined. Used in course outlines for an assessment date that has not yet been set.
UC	Use Case.
ULC	Unlikely Change.
WCAG	Web Content Accessibility Guidelines, the W3C standard referenced by AODA for web content [**? ?**].
Workload Cluster	A period of *CLUSTER_WINDOW* days in which one cohort has more than *CLUSTER_THRESHOLD* assessments due (Table 9).
Year Level	The level of a course, given by the first digit of its course number (for example, 2 for SFWRENG 2DA4).

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended readers of this Software Requirements Specification include project supervisors, CAS administrators, software developers, testers, and future maintainers. Technical readers should have a foundational understanding of software development, web applications, databases, and software testing. Familiarity with PDF processing and large language models (LLMs) is beneficial but not required. CAS administrators and project supervisors should be familiar with academic scheduling, course outlines, and relevant university policies, including CEAB requirements. This document is intended to communicate system requirements clearly to both technical and non-technical readers.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

The CAS Department at McMaster University currently manages course outlines and assessment schedules independently across courses. Without a centralized view of assessment dates, students in the same program and academic year may face multiple midterms and assignment deadlines within short periods. CAS administrative staff must also manually review course outlines for compliance with university policies and CEAB requirements. Missing or incomplete information, such as midterm dates listed as "TBD," creates additional work and requires repeated communication with instructors.

These challenges contribute to uneven student workloads, increased academic stress, and significant administrative overhead. Furthermore, instructors retain authority over their course schedules, limiting administrators' ability to directly resolve scheduling conflicts.

The CAS Deadline Management System aims to improve this process by centralizing assessment information, identifying potential scheduling conflicts and policy compliance gaps, and suggesting alternative assessment dates. This will help administrators coordinate schedules more efficiently, reduce manual review efforts, and support more balanced student workloads while preserving instructors' authority over their courses.

2.2 Product Vision

The CAS Deadline Management System will provide the CAS Department with a trusted, department-wide foundation for collaborative assessment planning. It will turn course-outline information into a consistent, reviewable source of scheduling knowledge that helps instructors and administrators make informed decisions together. The system is intended to improve coordination and transparency over time, while keeping final responsibility for course schedules and policy interpretation with the appropriate human decision-makers.

2.3 System Context

The CAS Deadline Management System is a web-based application intended to support course outline review, assessment scheduling, and policy compliance within the CAS Department at McMaster University. CAS administrators and course instructors interact with the system to upload course outlines, review extracted information, view assessment schedules, and evaluate potential scheduling conflicts and compliance issues. Figure 1 summarizes these interactions by showing the system boundary and its relationships with instructors, CAS administrators, the LLM, and the database.

The system processes course outlines and relevant academic scheduling information to extract assessment details, identify missing or non-compliant information, and recommend alternative assessment dates. Results are presented to users for review, while instructors and authorized administrators retain responsibility for final scheduling and compliance decisions.

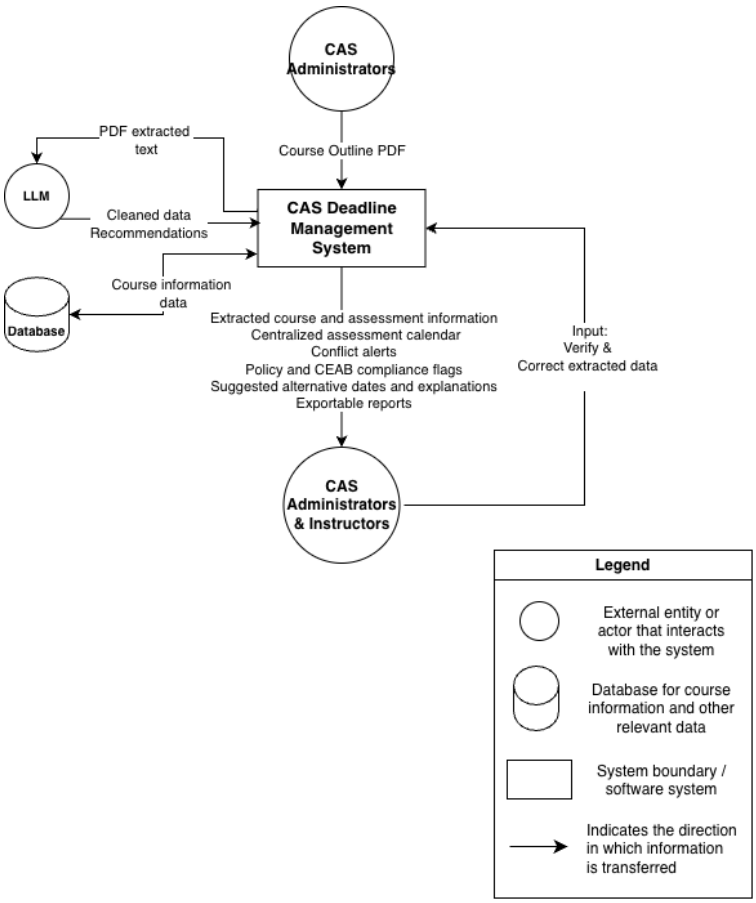


Figure 1: System Context

The system is intended for academic administration and planning rather than mission-critical or safety-critical applications. Since extracted information and automated recommendations may be incomplete or inaccurate, users must review flagged issues and verify important results before making decisions.

- **User Responsibilities:**
  - Provide course outlines and relevant academic scheduling information.
  - Ensure submitted documents are appropriate for review and that the information provided is sufficiently complete and accurate.
  - Review extracted course and assessment information and correct errors where supported.
  - Evaluate flagged scheduling conflicts, missing information, and potential policy compliance issues.

  - Decide whether to accept or reject suggested assessment dates and determine any final changes to course schedules.

- **Software Engineering Responsibilities:**
  - Accept and process supported course outline documents and scheduling information.
  - Extract and organize relevant course and assessment details for review.
  - Validate extracted data for missing, inconsistent, or incorrectly formatted values where possible.
  - Present assessment schedules and identify potential workload conflicts.
  - Flag potential gaps in course outline requirements and applicable accreditation criteria.
  - Generate alternative assessment date recommendations with supporting explanations.
  - Present results and warnings to users without automatically changing course outlines or assessment schedules.

- **External System Responsibilities:**
  - **LLM:** Process course outline content to support information extraction and, where applicable, the generation of explanations for scheduling recommendations.
  - **Database:** Store and retrieve course information, extracted assessment details, review results, and scheduling information.

2.4 User Characteristics

The primary users of the CAS Deadline Management System are CAS administrators and course instructors at McMaster University. Users are expected to have basic computer skills, including navigating web applications and viewing course information. CAS administrators should be familiar with course outline reviews, academic scheduling, and applicable university policies. Course instructors should have knowledge of their own course outlines and assessment schedules. No programming or machine learning knowledge is required. Users should be able to review extracted information, interpret flagged issues and scheduling recommendations, and make informed decisions regarding their courses or administrative responsibilities.

2.5 System Constraints

The CAS Deadline Management System must operate within the technical and institutional constraints of McMaster University. The following constraints apply to the system:

- **Deployment Environment:** The system must be deployable within the computing resources made available by McMaster University. The availability and capacity of these resources may limit the models and processing workloads the system can support.

- **Data Sources:** The system must process course outlines provided in PDF format. Variations in document structure, formatting, and scan quality may affect the accuracy of information extraction.

- **Language Model Cost and Hosting:** The system must use an LLM that is available at no cost to the project. GitHub Copilot should be preferred where it satisfies the system's technical and institutional requirements. Otherwise, the system should prioritize locally hosted, open-weight language models to minimize reliance on external services. Model selection must account for available computing resources and applicable licensing restrictions.

- **Human Decision-Making:** The system must not automatically modify course outlines or assessment dates. All scheduling changes and final compliance decisions remain the responsibility of instructors and authorized administrators.

- **Policy and Accreditation Requirements:** Compliance checks must be based on documented McMaster University policies [**?**] and applicable CEAB requirements [**?**]. The system must support human review of flagged issues rather than treating automated results as definitive determinations.

3 Stakeholder Requirements

SR1: **Centralized Course Outline Submission:** CAS administrators need a centralized location to collect and review course outlines.

SR2: **Course Information Extraction:** CAS administrators need relevant information extracted from submitted course outlines, including course details, assignments and projects, marking schemes, and midterm dates where available, and need to be able to export that information for use outside the system.

SR3: **McMaster Policy Review:** CAS administrators need support in identifying potential failures of course outlines to comply with McMaster University and CAS department policies.

SR4: **CEAB Accreditation Review:** CAS administrators need support in reviewing course outlines against CEAB accreditation requirements, including evaluating how each course addresses relevant CEAB graduate attributes.

SR5: **Assessment Schedule Analysis:** Professors/instructors and CAS administrators need to identify periods with high concentrations of assessment and midterm deadlines

across relevant CAS department courses and reassess the schedule when dates are updated.

SR6: **Scheduling Recommendations:** Professors/instructors need recommendations for alternative midterm and assessment dates that may reduce deadline bottlenecks for students, while being able to make the final decision.

SR7: **Instructor Decision-Making:** Professors/instructors need to retain authority over whether to accept or reject scheduling recommendations.

SR8: **Handling Incomplete Information:** Professors/instructors and CAS administrators need to be informed when missing or unreadable course outline information may affect the reliability of compliance reviews or scheduling recommendations.

SR9: **Authorized Access:** CAS administrators and professors/instructors need access to the information and functions relevant to their roles, while assuring sensitive information is protected and restricted to authorized users.

SR10: **Shared Assessment Calendar:** Professors/instructors and CAS administrators need access to a shared calendar displaying relevant course assessment dates.

SR11: **Exam and Restricted-Period Conflicts:** Professors/instructors and CAS administrators need to identify potential conflicts between assessments, exam dates, and applicable restricted periods.

4 User Personas

4.1 Persona 1

- **Name:** Bob M. Builder (CAS Administrator)
- **Description:** Bob is responsible for collecting and reviewing CAS course outlines to help ensure they align with McMaster University and CAS department policies, as well as CEAB accreditation requirements. Bob needs an efficient way to carry out his administrative duties while reducing the amount of redundant back and forth communication with instructors to ensure that outlines are aligned with the required policies and accreditation requirements.
- **Goals:**
  - To collect and review course outlines in one centralized location.
  - To identify missing information and potential policy or accreditation issues.
  - To reduce the time spent manually reviewing multiple course outlines.
  - To reduce unproductive communication time with course instructors.

- **Frustrations:**
  - Course outlines may use different formats or organize information differently.
  - Manually locating relevant information across many documents can be time-consuming.
  - Missing or unclear information can make it difficult to assess compliance.
- **Technical Ability:** Moderate (experience with administrative software, document management tools, spreadsheets)

4.2 Persona 2

- **Name:** Dora Sanchez Explora (Software Engineering Professor)
- **Description:** Dora teaches Software Engineering Design courses and plans assignments and two midterms for the semester. She wants to provide students with a workload that is easy to manage and deadlines that allow them to learn and adequately prepare for their midterms without overwhelming them in their other courses.
- **Goals:**
  - To submit and maintain tentative assessment dates efficiently.
  - To identify potential conflicts with assessments in other courses.
  - To receive practical recommendations for distributing assessment deadlines more evenly.
  - To retain final authority over assessment dates for her course.
- **Frustrations:**
  - Coordinating assessment dates across courses can be difficult.
  - Several courses may schedule major assessments around the same time.
  - Changing an assessment date may create conflicts with other course activities or require additional coordination.
- **Technical Ability:** Moderate (experience with administrative management systems, online course and university tools)

4.3 Persona 3

- **Name:** Homer Simpson (4th Year Software Engineering Student)
- **Description:** Homer is an undergraduate Software Engineering student taking six courses for the Fall term. He is feeling overwhelmed by the workload and is struggling to manage his time between getting his work done and studying for his upcoming back-to-back midterms.

- **Goals:**
  - To have a more balanced distribution of assessments throughout the term.
  - To avoid periods when multiple major deadlines occur close together.
  - To have sufficient time to prepare for major assessments across different courses.
- **Frustrations:**
  - Multiple assignments and midterms may fall within the same few days.
  - Heavy assessment periods can make it difficult to allocate study/work time effectively.
  - Assessment dates across different courses may be difficult to coordinate.
- **Technical Ability:** High (experience with online learning platforms, digital calendars, student productivity tools and software)

5 Use Cases

5.1 UC1: Submit Course Outline

Primary Actor: CAS Administrator
Preconditions

- The administrator has access to the system.
- A CAS course outline is available for submission.

Main Success Scenario

1. The administrator logs into the system.
2. The administrator selects the option to submit a course outline.
3. The administrator selects and submits the course outline.
4. The system receives the document and attempts to extract relevant course information.
5. The system confirms that the course outline has been submitted successfully.
6. The system makes the submitted outline available for information extraction and review.

Alternative Flows

- If the submitted document cannot be read, the system informs the administrator that a readable document is required.

- If the submission fails, the system informs the administrator and allows them to try again.
- If relevant course information is missing, the system proceeds with the information that is available and identifies the missing information.
- If the Copilot or LLM service is unavailable, the system informs the administrator that automatic information extraction cannot be completed. The submitted course outline remains available, and the administrator can retry extraction when the service becomes available.

Postconditions

- If submission succeeds, the course outline is stored and available for review, even if automatic information extraction could not be completed.
- If submission fails, the administrator is informed that the course outline was not submitted successfully.

5.2 UC2: Review Extracted Course Information

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted successfully.
- The system has attempted to extract relevant course information.

Main Success Scenario

1. The administrator selects a submitted course outline.
2. The system presents the extracted course details and information, including assignments, marking schemes, and midterm dates where available, and flags any missing, uncertain, or ambiguous information for review.
3. The administrator reviews the extracted information against the original course outline.
4. The administrator fills in any missing information or confirms that the information is correct.
5. The system saves the verified course information for later steps and scheduling analysis.

Alternative Flows

- If the system cannot extract certain information, it identifies the missing information for the administrator to review.

- If the system extracts information that is uncertain or ambiguous, it flags the affected information for administrator review and indicates that verification is required.
- If the administrator identifies incorrect information, they correct it before proceeding.
- If the administrator needs the information outside the system, they export the verified course information as a CSV file.

Postconditions

- The extracted course information has been reviewed and corrected where necessary.

5.3 UC3: Review Course Outline Against McMaster and CAS Policies

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted.
- Relevant course information is available for review.
- McMaster University and CAS department policy criteria are available to the system.

Main Success Scenario

1. The administrator selects a course outline for policy review.
2. The system evaluates the available course information against policy criteria.
3. The system identifies potential policy inconsistencies or missing required information.
4. The system presents the findings and supporting information to the administrator.
5. The administrator reviews the findings and determines which issues require further investigation or correction.

Alternative Flows

- If relevant information is missing, the system identifies the limitation and presents any findings that can still be supported.
- If a potential policy issue cannot be determined conclusively, the system flags it for manual review.
- If the necessary policy criteria are unavailable, the system informs the administrator that the review cannot be completed reliably.

Postconditions

- The administrator has access to policy review findings and can identify issues requiring further attention.

5.4 UC4: Review Course Outline Against CEAB Criteria

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted.
- Relevant course information is available for review.
- Applicable CEAB accreditation criteria are available to the system.

Main Success Scenario

1. The administrator selects a course outline for CEAB-related review.
2. The system evaluates the available course information against CEAB criteria.
3. The system identifies potential information gaps that may require further examination.
4. The system presents the findings and supporting information to the administrator.
5. The administrator reviews the findings and determines which issues require further investigation or correction.

Alternative Flows

- If relevant information is missing, the system identifies the limitation and presents any findings that can still be supported.
- If a potential gap requires professional judgment, the system flags it for manual review.
- If the necessary CEAB criteria are unavailable, the system informs the administrator that the review cannot be completed reliably.

Postconditions

- The administrator has access to CEAB-related review findings and can identify issues requiring further attention.

5.5 UC5: Review Course Outline Findings

Primary Actor: CAS Administrator
Preconditions

- A course outline has been submitted.
- At least one policy or CEAB-related review has been performed.

Main Success Scenario

1. The administrator opens the review findings for a course outline.
2. The system displays identified potential issues, missing information, and relevant supporting details.
3. The administrator examines the findings and determines required follow-up.
4. The administrator uses the findings to support the course outline review process.

Alternative Flows

- If no potential issues are identified, the system indicates that no issues were detected by the review.
- If the findings depend on incomplete or uncertain information, the system identifies those limitations.

Postconditions

- The administrator has reviewed the available findings and identified any issues requiring follow-up.

5.6 UC6: Submit Tentative Assessment Dates

Primary Actor: Professor/Instructor
Preconditions

- The professor or instructor has access to the system.
- The instructor has tentative assessment dates for a relevant course.

Main Success Scenario

1. The instructor selects the relevant course.
2. The instructor enters or submits tentative dates for assessments and midterms.
3. The system checks the submitted information for completeness.
4. The system presents any identified missing or invalid information to the instructor.
5. The instructor corrects any identified issues and submits the information.
6. The system saves the tentative assessment dates for scheduling analysis.

Alternative Flows

- If required assessment information is missing, the system identifies the missing details and asks the instructor to complete them.

- If a date is invalid or cannot be interpreted, the system asks the instructor to correct it.
- If the instructor needs to revise the submitted dates, they can update the information and submit it again.

Postconditions

- The instructor's tentative assessment dates are stored and available for scheduling analysis.

5.7 UC7: Analyze Assessment Schedule

Primary Actor: Professor/Instructor, CAS Administrator
Preconditions

- The user has access to the system.
- Assessment dates are available for the relevant courses, if any have been submitted.

Main Success Scenario

1. The instructor requests an analysis of the assessment schedule.
2. The system gathers the available assessment dates for relevant courses.
3. The system analyzes the distribution of assessments and midterms across the relevant period.
4. The system identifies periods with high concentrations of assessments or potential student workload bottlenecks.
5. The system presents the analysis results to the instructor.

Alternative Flows

- If assessment information is incomplete, the system identifies the missing information and explains how it limits the analysis.
- If insufficient assessment dates are available, the system informs the instructor that a reliable analysis cannot be completed.
- If no significant bottlenecks are identified, the system presents the analysis without recommending unnecessary changes.
- If the instructor or administrator wants to view the assessment calendar without performing an analysis, they can filter the calendar by academic year and program. The system displays the assessment dates matching the selected filters without generating workload analysis or recommendations.

Postconditions

- The instructor or administrator can review the assessment schedule, either through the calendar view or through the results of a schedule analysis, depending on the requested operation.

5.8 UC8: Generate Scheduling Recommendations

Primary Actor: Professor/Instructor
Preconditions

- An assessment schedule analysis has been completed.
- Sufficient information is available to identify potential alternative dates.

Main Success Scenario

1. The instructor requests scheduling recommendations based on the available assessment schedule.
2. The system identifies assessments that contribute to periods with high concentrations of deadlines.
3. The system evaluates potential alternative dates for affected assessments.
4. The system generates recommendations intended to reduce assessment bottlenecks while considering the available scheduling information.
5. The system presents the suggested dates and the available reasoning behind the recommendations.
6. The instructor reviews the recommendations.

Alternative Flows

- If no suitable alternative dates can be identified, the system informs the instructor that it cannot provide a suitable recommendation.
- If relevant scheduling information is missing, the system identifies the limitation.
- If several alternatives are available, the system presents the alternatives for the instructor to evaluate.
- If the LLM/Copilot service goes down, the system notifies the instructor, prevents showing false explanations, and allows retrying once the service recovers.

Postconditions

- Scheduling recommendations (if applicable) are presented to the instructor for consideration. No assessment dates are changed automatically.

5.9 UC9: Review and Decide on Scheduling Recommendations

Primary Actor: Professor/Instructor
Preconditions

- Scheduling recommendations have been generated for the relevant assessment schedule.

Main Success Scenario

1. The instructor opens the scheduling recommendations.
2. The system displays the suggested assessment dates and the rationale for each recommendation.
3. The instructor evaluates the suggestions against course-specific constraints and other relevant considerations.
4. The instructor chooses to accept, reject, or modify a proposed date.
5. The system records the instructor's decision and updates the schedule if an accepted or modified date is submitted.

Alternative Flows

- If the instructor rejects a recommendation, the existing assessment date remains unchanged.
- If the instructor modifies a recommendation, the system allows the revised date to be submitted.
- If the instructor needs more information, they can review the schedule analysis before making a decision.

Postconditions

- The instructor's decision is reflected in the schedule where an update has been submitted. Rejected recommendations do not change the existing assessment dates.

5.10 UC10: Update Assessment Dates and Re-analyze Schedule

Primary Actor: Professor/Instructor
Preconditions

- Assessment dates for the relevant course have previously been submitted.
- The instructor has identified a need to change one or more tentative assessment dates.

Main Success Scenario

1. The instructor selects the relevant course and assessment schedule.
2. The instructor updates one or more tentative assessment dates.
3. The system checks the updated information for completeness and validity, then saves the updated dates.
4. The instructor requests a new assessment schedule analysis.
5. The system analyzes the updated schedule and identifies any remaining or newly introduced bottlenecks.
6. The system presents the updated analysis and any resulting recommendations.

Alternative Flows

- If an updated date is invalid or incomplete, the system asks the instructor to correct it before proceeding.
- If insufficient information is available for re-analysis, the system identifies the missing information.
- If the updated schedule introduces new bottlenecks, the system identifies them in the new analysis.

Postconditions

- The updated tentative assessment dates are available in the system, and a new analysis is available if sufficient information was provided.

6 Functional Requirements

The system shall support the following capabilities, grouped by feature area.

Outline Intake and Extraction

FR1: The system shall accept course outlines in PDF format submitted by a CAS administrator or professor.

FR2: The system shall extract each assessment's name, type, date, and weight from a course outline.

FR3: The system shall extract the learning outcomes in a course outline and the graduate attributes each addresses, whether the outline lists them by number, names them, or only describes them.

FR4: The system shall flag any extracted field that is missing, including a date that is marked TBD.

FR5: The system shall require a user to review and confirm extracted data before it is used for the calendar, conflict detection, or compliance checking.

FR6: The system shall allow an instructor to enter and update tentative assessment dates for their own courses.

Calendar and Conflict Detection

FR7: The system shall display confirmed and tentative assessment dates in a shared calendar that can be filtered by course, year, and program, with tentative dates marked as such.

FR8: The system shall identify weeks in which a student cohort has more assessments than a configurable threshold.

FR9: The system shall identify assessments scheduled in periods restricted by McMaster policy [**?**], such as final exam periods.

FR10: The system shall re-run conflict detection for the affected courses and cohorts whenever an assessment date is added or changed.

Scheduling Suggestions

FR11: The system shall, on explicit request by the instructor, suggest alternative dates, for assessments involved in a detected conflict or listed as TBD.

FR12: The system shall not change any assessment date without the explicit approval of the responsible instructor.

Compliance Checking

FR13: The system shall check each outline against the McMaster course outline policy rules, including required sections and assessment weights totalling 100%, and report each finding [**?**].

FR14: The system shall compare the graduate attribute numbers in an outline with the master course sheet and flag any that are missing or inconsistent.

Access and Export

FR15: The system shall require users to log in before they can view or change any data.

FR16: The system shall allow administrators to manage data for all courses, and shall allow instructors to change data only for their own courses.

FR17: The system shall allow an administrator to export extracted assessment data as a CSV file.

7 Data Requirements

7.1 Major Data Entities

- **User:** an administrator or instructor. Both can submit outlines and confirm extracted data. Instructors also enter tentative dates, and request and approve date suggestions, for their own courses.
- **Course:** a CAS course offering in a given term, taught by one instructor. A course may be cross-listed. SFWRENG 4X03, COMPSCI 3X03, and MECHTRON 3X03 are one course taught by one instructor, with a separate course code for each program. The outline and its assessments are stored once.
- **CourseCode:** one listing of a course. Each code belongs to one course, and a course has one or more codes. The program and year level are derived from the code and are not entered separately.
- **CourseOutline:** a submitted course outline PDF. Each outline belongs to one course, including when that course has several course codes.
- **Assessment:** a graded component of a course (midterm, assignment, lab, etc.) with a weight and a date. The date is empty when it is missing or TBD, and an instructor can enter a tentative date.
- **LearningOutcome:** a learning outcome stated in an outline, with any graduate attribute numbers listed beside it. A user reviews each outcome after extraction, and the review status records whether it is still extracted or has been confirmed.
- **MasterCourseRecord:** a row of the master course sheet, listing the graduate attribute numbers expected for a course code. A cross-listed course can match more than one row, one for each of its codes.
- **AcademicCalendarEntry:** a holiday, exam period, or other restricted date range.
- **DateRecommendation:** a suggested alternative date for an assessment, with its explanation and decision.
- **ComplianceFinding:** a problem found when checking an outline against McMaster policy or the master course sheet.

An instructor teaches one or more courses. Each course has one or more course codes and one or more outlines. Each outline contains many assessments and learning outcomes, stored

once for the course rather than once per course code. Student cohorts are found by grouping course codes that share the same derived program and year level. Workload conflicts are calculated from the stored assessments when needed, so neither cohorts nor conflicts are stored separately.

7.2 Data Dictionary

Entity	Attributes (type): description
User	user_id (UUID): unique identifier. email (String): user's email address. role (Enum): administrator or instructor.
Course	course_id (UUID): unique identifier. instructor_id (UUID): user who teaches the course. term (String): academic term of the offering.
CourseCode	course_code_id (UUID): unique identifier. course_id (UUID): course this listing belongs to. code (String): course code, e.g., SFWRENG 4X03. program (String, derived): program inferred from the subject in the code, for example Software Engineering from SFWRENG. Not entered separately. year_level (Integer, derived): program year inferred from the first digit of the course number, for example 4 in SFWRENG 4X03 and 3 in COMPSCI 3X03. Not entered separately.
CourseOutline	outline_id (UUID): unique identifier. course_id (UUID): course the outline belongs to. submitted_at (Timestamp): when the outline was submitted.
Assessment	assessment_id (UUID): unique identifier. outline_id (UUID): outline the assessment was extracted from. name (String): name as written in the outline. type (Enum): midterm, assignment, quiz, lab, project, final exam, other. due_date (Date or null): null when missing or TBD. is_tentative (Boolean): true when the date was entered by an instructor and is not final. weight_percent (Decimal): percentage of the final grade. review_status (Enum): extracted or confirmed. source_text (String): outline text the values were extracted from.

Entity	Attributes (type): description
LearningOutcome	outcome_id (UUID): unique identifier. outline_id (UUID): outline the outcome appears in. text (String): outcome as written. ga_numbers (List of Integer): graduate attribute numbers listed in the outline. review_status (Enum): extracted or confirmed. source_text (String): outline text the values were extracted from.
MasterCourseRecord	course_code (String): course code this row applies to. Matches a CourseCode. expected_ga_numbers (List of Integer): attribute numbers the master sheet lists for that course code.
AcademicCalendarEntry	start_date (Date): start of the range. end_date (Date): end of the range. restriction (Enum): holiday, exam period, no-assessment period.
DateRecommendation	recommendation_id (UUID): unique identifier. assessment_id (UUID): assessment the suggestion applies to. suggested_date (Date): proposed date. explanation (String): reason for the suggestion, shown to the instructor. decision (Enum): pending, accepted, rejected.
ComplianceFinding	finding_id (UUID): unique identifier. outline_id (UUID): outline that was checked. message (String): description of the problem found.

Table 2: Data Dictionary

7.3 Privacy and Data Retention

DR1: Course outlines and the data extracted from them are generally public. The master course sheet and other department data may be confidential, and shall only be available to authorized CAS users.

DR2: The system shall use GitHub Copilot as its language model where it meets the system's requirements, and a locally hosted open-weight model on CAS infrastructure otherwise. Copilot is an external service, so only publicly available course outline information may be sent to it. Confidential department data and student personal information shall not be sent to Copilot or to any other external service. All other course and department data shall be stored and processed on CAS-hosted infrastructure.

DR3: Users shall only see and change the data their role allows.

DR4: The retention period for outlines, extracted data, and findings shall be 10 years. The Undergraduate Course Management Policy requires that course outlines be retained for at least 10 years [**?**].

8 Non-Functional Requirements

NFR1: **Accuracy:**

- The system shall achieve at least 95% accuracy for extracted assessment dates and weights on a representative test dataset of course outlines.
- The system shall achieve at least 95% recall for predefined missing or incomplete information in the same test dataset.
- The system shall identify at least 95% of predefined policy and CEAB compliance issues and report the results in the Verification and Validation Plan.
- These thresholds align with the success measures in the Problem Statement and Goals document [**?**].

NFR2: **Usability:**

- The system shall allow the CAS administrators and course instructors described in Section 2.4 to upload course outlines, review extracted information, view assessment schedules, and interpret flagged issues without programming or machine learning knowledge.
- At least 80% of participants in representative usability testing shall complete each designated core task without assistance.
- The 80% target is a project-defined validation threshold intended to demonstrate that the core workflows are usable by the intended non-technical users.

NFR3: **Maintainability:**

- The system shall be organized into modular components for document processing, information extraction, scheduling analysis, and compliance rules.
- Changes to one of these components shall not require unrelated components to be rewritten.
- For representative maintenance tasks defined in the Verification and Validation Plan, the effort required to implement each change shall be documented and assessed against the original implementation effort for the affected component.

NFR4: **Portability:**

- The system shall be deployable in the designated McMaster University hosting environment.

- The system shall be usable through Google Chrome 120 or later, Mozilla Firefox 121 or later, and Safari 17 or later on supported desktop operating systems.
- The deployment and core workflow tests defined in the Verification and Validation Plan shall be executed in each supported browser environment. These browser versions define the project's verification baseline and may be updated for a future release.

NFR5: **Installability and Deployability:**

- A new maintainer shall be able to deploy the system to the designated McMaster University hosting environment by following the documented installation and deployment instructions.
- The documented procedure shall identify required software, configuration values, environment variables, database setup, and service startup steps without relying on undocumented knowledge.
- The deployment shall be verified by a deployment test in the Verification and Validation Plan, including confirmation that the deployed system supports the core workflows.

NFR6: **Performance:**

- Under the test environment and workload defined in the Verification and Validation Plan, the system shall display previously processed assessment calendar data within 3 seconds for at least 95% of requests.
- For a supported course outline of up to 30 pages, the system shall complete document processing and information extraction within 60 seconds for at least 90% of test runs, excluding failures or delays caused by unavailable external dependencies.
- The test environment, document characteristics, workload, and response-time results shall be recorded.
- The 3-second target is a project-defined interactive response target, while the 60-second target bounds processing time for a typical supported outline; both targets shall be validated under the controlled conditions defined in the Verification and Validation Plan.

NFR7: **Reliability:**

- The system shall handle invalid, incomplete, or unsupported input documents without crashing or corrupting previously stored information.
- All predefined invalid-input cases shall produce an appropriate error message, and previously stored records shall remain accessible after a failed upload or processing attempt.

- If the LLM or Copilot service is unavailable, the system shall continue to support viewing and editing stored course and assessment information, viewing the assessment calendar, and exporting available data.
- Submitted course outlines shall be retained, and users shall be able to retry extraction or recommendations when the service becomes available, as described in UC1 and UC8.

NFR8: **Security:**

- The system shall implement the access controls specified in FR15 and FR16, together with the privacy and data-retention requirements in Section 7.3.
- Security verification shall include tests for unauthorized access, role-based permissions, and the protection of sensitive configuration values and credentials.

NFR9: **Accessibility:**

- The system's web interface shall conform to the applicable requirements of the Accessibility for Ontarians with Disabilities Act (AODA) and WCAG 2.0 Level AA [**? ? ?**].
- Core workflows, including authentication, course-outline submission, information review, calendar viewing, and data export, shall be operable using keyboard navigation and shall provide accessible labels, headings, focus indicators, and error messages.
- Accessibility verification shall include an automated accessibility scan and manual keyboard and screen-reader checks for the core workflows. The applicable WCAG success criteria and test results shall be recorded in the Verification and Validation Plan.

9 AI and Machine Learning Requirements

The system extracts information from course outline PDFs in two steps. First, a Python PDF text extraction library reads the text from each PDF. Then a large language model (LLM) cleans up that text and organizes it into structured data. Where an outline does not list graduate attribute numbers or their exact wording, the LLM identifies the attributes from the learning outcome text. The attributes are then compared with the master course sheet by the system's own code, so the confidential master course sheet is never sent to the LLM. The date suggestions come from a constraint solver (CP-SAT), and the assessment weight check is ordinary deterministic code, so neither uses the LLM. The preferred LLM is GitHub Copilot, with a locally hosted open-weight model on CAS infrastructure as the fallback. The LLM is kept behind a replaceable interface, so the model can be changed without changing the rest of the system. Copilot is an external service, and the only model input that may be sent to it is publicly available course outline text (DR2).

9.1 Model Inputs

AIR1: The model shall receive the text that the PDF extraction library produced from each course outline, rather than the PDF file itself, together with the CEAB graduate attribute definitions [**?**]. The master course sheet shall not be part of the model input (DR1, DR2).

AIR2: Model input shall be sent only to GitHub Copilot or to a model hosted on CAS-operated infrastructure (DR2). Only publicly available course outline text may be sent to Copilot. No student personal information shall be used as model input.

9.2 Model Outputs

AIR3: The model shall organize the extracted outline text into structured data containing, for each assessment, its name, type, date, and weight, and for each learning outcome, its text, the graduate attributes it addresses, and whether each attribute was listed in the outline or identified by the model (FR2, FR3).

AIR4: Every structured value shall include the passage of extracted text it was taken from, so a reviewer can check it without opening the original document.

AIR5: The model shall only use information present in the extracted text, and shall report a value as missing, including a date that is marked TBD or is too vague to resolve, instead of inventing one (FR4).

AIR6: No model output shall be used for the calendar, conflict detection, or compliance results until a user has confirmed it (FR5).

9.3 Evaluation Metrics

Each metric shall be measured on a set of sample course outlines, using an answer key prepared manually by the team as the reference. Extraction metrics are measured on the final structured output, so they cover the PDF library and the model together.

Metric	**What it measures**	**Target**
Extraction accuracy	Share of assessment dates and weights that are correct in the final structured output (FR2)	At least 95%
Missing-field detection	Share of missing or TBD fields that are flagged (FR4)	At least 95%
Compliance issue detection	Share of known compliance issues that are found (FR13, FR14)	At least 95%

Table 3: Evaluation Metrics

9.4 Known Limitations

The table lists the expected failure modes and how the design contains each one. The main safeguard is that a person confirms every extracted value, with the source text shown beside it. During evaluation, the team also manually reviews a sample of the outputs.

Failure mode	**Effect**	**Mitigation**
PDF library returns messed up or incomplete text (tables, multiple columns, scanned pages)	Dates, weights, or sections are missing or out of order before the model sees them	Flag outlines with little or no extracted text for manual entry; test on outlines with a range of layouts
Model changes or adds values while cleaning up the text	A date or weight that differs from, or appears nowhere in, the outline	Show the source text; reject values with no source passage; require review
Ambiguous dates, such as "week of Oct 20" or "TBD"	A date cannot be resolved without a person	Report the date as missing and never guess
Reviewer over-trusts the output	Wrong values are confirmed without being checked	Show the source beside each value; confirm explicitly, never by default
Courses missing from the master sheet (technical electives, mechatronics, computer science)	Attribute comparison is not possible	Tell the user the course was not checked, rather than showing no findings
Uneven accuracy across layouts or programs	Some instructors see more errors than others	Test on outlines from each program; add examples from weak areas
CEAB requirements are not exhaustive	A clean result does not mean full accreditation compliance	State that only the encoded rules were checked
LLM unavailable	Extraction and scheduling explanations cannot be completed	Inform the user, keep the submitted outline, do not present an explanation that was not generated, and allow a retry when the service returns (UC_CAS_01, UC_PROF_03)

Table 4: Expected Failure Modes

10 Requirements Traceability Matrices

Stakeholder Req.	**Use Case(s)**	**Functional Req(s).**
SR1	UC1	FR1
SR2	UC1, UC2	FR2, FR3, FR5, FR17
SR3	UC3, UC5	FR13
SR4	UC4, UC5	FR3, FR14
SR5	UC6, UC7, UC10	FR6, FR8, FR10
SR6	UC8	FR11
SR7	UC9	FR12
SR8	UC2, UC6, UC7	FR4, FR5
SR9	UC1, UC6	FR15, FR16
SR10	UC7	FR7
SR11	UC7	FR9

Table 5: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req.	**NFR1** Acc.	**NFR2** Usab.	**NFR3** Maint.	**NFR4** Port.	**NFR5** Inst.	**NFR6** Perf.	**NFR7** Rel.	**NFR8** Sec.	**NFR9** Access.
FR1 (outline intake)			✓			✓	✓		✓
FR2 (assessment extraction)	✓		✓			✓			
FR3 (outcome extraction)	✓		✓						
FR4 (flag missing/TBD)	✓						✓		
FR5 (user confirmation)	✓	✓							✓
FR6 (tentative dates)		✓					✓	✓	✓
FR7 (shared calendar)		✓		✓		✓			✓
FR8 (workload clusters)			✓						
FR9 (restricted periods)			✓						
FR10 (re-analysis)		✓							
FR11 (date suggestions)		✓	✓						✓
FR12 (no auto-change)								✓	
FR13 (policy check)	✓		✓						
FR14 (graduate attributes)	✓		✓						
FR15 (log-in)								✓	✓
FR16 (role-based access)								✓	
FR17 (CSV export)		✓						✓	✓

Table 6: Traceability Between Functional and Nonfunctional Requirements

NFR5 has no entries in Table 6 because it applies to the system as a whole rather than to any one functional requirement. It is verified by the deployment test in the Verification and Validation Plan.

Likely Change	Affected Requirements and Assumptions
LC1	FR2, FR3, FR11, NFR6, NFR8, DR2, AIR2, A5
LC2	FR1, FR2, FR3, FR4, NFR6, AIR1
LC3	FR13, FR9
LC4	SR4, FR3, FR14, A7
LC5	FR3, FR14, AIR1, AIR3
LC6	FR8, FR10, FR11
LC7	FR8, A1, A3
LC8	FR9, A6
LC9	FR11
LC10	FR11, FR13, FR14
LC11	FR17
LC12	SR9, FR15, NFR8
LC13	FR8, A1, A3
LC14	SR1, FR7, FR16, A1

Table 7: Traceability Between Likely Changes and Requirements

AI or Data Requirement	Related Requirements
AIR1	FR2, FR3
AIR2	DR2
AIR3	FR2, FR3
AIR4	FR5
AIR5	FR4
AIR6	FR5
DR1	FR16
DR2	AIR2, NFR8
DR3	FR16
DR4	NFR8

Table 8: Traceability Between AI Requirements, Data Requirements, and Functional Requirements

11 Rationale

This section explains the major decisions in this document that are not obvious. It also states the assumptions that the requirements rely on (Section 11.1). Each item references the

requirement, constraint, or assumption it justifies. The sources are the information session and meetings with the CAS supervisors, the project's Problem Statement and Goals [**?**] and Development Plan [**?**], McMaster University's Undergraduate Course Management Policy [**?**], and the CEAB Accreditation Criteria and Procedures [**?**].

11.1 Assumptions

The requirements in this document rely on the following assumptions. Each one narrows the scope of the system. If an assumption stops holding, the requirements that reference it must be revisited.

A1: **Program and year level come from the course code.** The CAS department provides the list of course codes it offers. The subject prefix of a course code identifies the program (for example, SFWRENG, COMPSCI, MECHTRON), and the first digit of the course number identifies the year level (for example, SFWRENG 2DA4 is a second-year course). A cohort is the set of courses that share a program and year level. This replaces timetable data, which the system does not use.

A2: **Cross-listed course codes refer to one course.** A course offered under several codes (for example, SFWRENG 4X03, COMPSCI 3X03 and MECHTRON 3X03) has one outline, one instructor and one set of assessments. Its assessments count once toward each cohort that one of its codes belongs to. A single course can therefore belong to cohorts in different years.

A3: **All courses at the same year level belong to one cohort, including electives.** Elective enrolment data is not available, and the CAS department has not yet said how electives should be grouped. Treating all same-level courses as one cohort may flag clusters that no individual student experiences. This is a conservative choice: it may produce false positives, but it does not hide real clusters, and users review every flag. This assumption is provisional (LC7).

A4: **Students taking courses outside their year level are not modelled.** A student who takes a higher-level course early, or a course out of sequence, accepts the additional workload. This is the student's responsibility, not the department's.

A5: **Course outlines are public documents.** Outlines must be available to students [**?**], so their content can be sent to a language model service covered by McMaster's institutional agreement [**?**]. Other data held by the system, such as compliance findings and tentative dates, is not public.

A6: **Academic calendar dates are available before analysis.** Term start and end dates, recesses, and examination periods for the term being analyzed are published by the university and entered into the system before workload analysis is run.

A7: **The department provides the expected graduate attributes for each course.** The master course sheet used by FR14 is supplied and maintained by the CAS department. The system does not decide which graduate attributes a course should cover.

11.2 System Constraints

- **Language model cost and hosting.** The client does not want to pay for ongoing services, so the language model must be free to the project. GitHub Copilot is preferred because McMaster's agreement with Microsoft provides it at no cost with enterprise data protection, meaning prompts are not used to train models [**?**]. A locally hosted open-weight model is the fallback if programmatic access to Copilot is not available. The final choice has not been made (LC1).

- **PDF input only.** The CAS supervisors stated that outlines published in other formats, such as HTML, are converted to PDF before they are submitted. Supporting one format keeps the extraction pipeline simpler to build and test (FR1).

- **Human decision-making** (SR7, FR12). Instructors have authority over course delivery, and CAS administrators are not authorized to reassign assessment dates. The CAS supervisors stated that the project must not change course content or dates. This is a constraint, not a preference: the university, not the team, decides who may change assessment dates.

- **Compliance grounded in documented policy.** Compliance checks are limited to rules stated in the Undergraduate Course Management Policy [**?**] and the CEAB Accreditation Criteria and Procedures [**?**]. Rules the team inferred itself could flag outlines incorrectly and could not be justified to instructors.

11.3 Functional Requirements

- **Extracted data must be confirmed before use** (FR5, SR2, SR8). Course outlines do not follow a common format, so extraction errors are expected. An incorrect date or weight would affect the calendar, workload analysis, suggestions, and compliance checks. Administrators need extracted information they can rely on (SR2), and users need to know when it may be unreliable (SR8). A confirmation step meets both needs by stopping an error before it reaches those outputs. The Development Plan [**?**] identifies this semi-automated approach as the fallback if fully automated extraction is not accurate enough.

- **Missing and TBD values are flagged, not guessed** (FR4, SR8). The CAS supervisors identified outlines listing "Midterm Date: TBD" as a core problem. A guessed date would appear in the calendar as if it were real. Flagging the value makes the gap visible so the administrator can follow up once, instead of repeatedly.

- **Instructors can enter tentative dates** (FR6). Outlines are often written before every date is fixed, which is why TBD dates occur. Letting instructors enter a tentative date gives the workload analysis something to work with, and marking it as tentative in the calendar (FR7) keeps it from being mistaken for a final date.

- **Clusters use symbolic constants** (FR8). There is no agreed definition of how many assessments in how many days is too many. The window and threshold are therefore the symbolic constants *CLUSTER_WINDOW* and *CLUSTER_THRESHOLD* (Table 9), so they can be tuned with the CAS supervisors without changing the requirement (LC6).

- **Restricted periods come from university policy** (FR9, SR11). The Undergraduate Course Management Policy forbids tests or examinations worth more than *BAN_WEIGHT_LIMIT* during the test and examination ban period. That period covers the last *BAN_CLASS_DAYS* days of classes and the days before final examinations begin [**?**]. Assessments may also not be due during recesses [**?**]. These are the restricted periods the system checks.

- **Conflicts are re-checked automatically; suggestions are not** (FR10, FR11). Detecting conflicts only reports facts about the stored dates, so it is re-run whenever a date changes to keep the calendar current. Suggestions are produced only when an instructor asks for them. The CAS supervisors noted that instructors already find unrequested follow-up disruptive, and unprompted suggestions would add to it.

- **Every suggestion comes with an explanation** (FR11, UC8). Instructors keep final authority, so a suggestion is only useful if the instructor can judge it. An explanation that compares the workload in the original and suggested weeks lets the instructor check the suggestion against the data. If the explanation cannot be generated, the suggested dates are not shown with a made-up reason (UC8).

- **Policy rules the compliance check uses** (FR13, SR3). The rules are taken from the Undergraduate Course Management Policy [**?**]:

  - assessment weights must total *WEIGHT_TOTAL*;
  - without approval, no single assessment may be worth more than *MAX_SINGLE_WEIGHT*;
  - students must receive feedback worth at least *EARLY_FEEDBACK_L2* (Level II) or *EARLY_FEEDBACK_L3* (Level III and above) before the last day to cancel a course.

  Weights that do not total *WEIGHT_TOTAL* are either an outline error or an extraction error, and both need review.

- **Access is limited by role** (FR15, FR16, SR9). Instructors may change only their own courses because they hold authority over those courses and no others. Administrators review outlines across the department, so they need access to all courses.

- **Export as CSV** (FR17). The CAS supervisors asked that stored data be exportable. CSV opens in the spreadsheet tools CAS staff already use and does not tie the data to this system. Other formats may be added later (LC11).

11.4 Nonfunctional Requirements

- **Accuracy targets** (NFR1). The Problem Statement and Goals [**?**] set extraction accuracy, missing-field recall and compliance detection at 90%. The team raised *EXTRACT_ACCURACY*, *FLAG_RECALL* and *COMPLIANCE_RECALL* to 95% while writing this document. Every extracted value is confirmed by a person before use (FR5), so the system is only useful if review is mostly confirmation rather than correction. At 90%, an administrator would correct roughly one value in ten, which would leave much of the manual effort the system is meant to remove. The targets are below 100% because review remains the safeguard. The Problem Statement and Goals [**?**] will be updated to match.

- **Maintainability is measured against likely changes** (NFR3). Section 12 lists the changes the team expects. Measuring maintainability as the effort to make one of those changes, relative to *FRACTION* of the original development effort, gives a target that can be tested rather than a general statement.

- **Performance and usability targets** (NFR6, NFR2). *CALENDAR_LOAD_TIME* applies to stored data that users browse interactively, so it is short. *EXTRACT_TIME_LIMIT* is longer because extraction runs once per outline when it is submitted, and the administrator reviews the result afterwards. These values, together with *TASK_COMPLETION*, are initial targets set by the team. They will be validated in the Proof of Concept and revised if they prove unrealistic.

- **Installability** (NFR5). The system is expected to outlive the capstone team, and will be maintained by CAS IT staff or a later student team. Deployment therefore cannot depend on knowledge that only the current team holds, so every step must be documented and tested.

- **Accessibility** (NFR9). The Accessibility for Ontarians with Disabilities Act [**?**] applies to McMaster's web content, and its standard for websites is WCAG 2.0 Level AA [**?** **?**]. This is a legal requirement, not a design preference, so it is stated as a nonfunctional requirement that must be verified.

11.5 Data Requirements

- **One record per cross-listed course** (A2). Storing a cross-listed course once means a correction made under one course code applies under all of them. It also prevents the same assessment from being counted twice in one cohort's workload.

- **No student personal data.** Cohorts are derived from course codes (A1), so the system never needs to know which students take which courses. This reduces privacy risk and the amount of confidential data the system holds.

11.6 AI and Machine Learning

- **Suggested dates are computed by a deterministic solver, not generated by the language model** (FR11). A language model can produce plausible but incorrect dates. A solver that works directly from the stored constraints produces dates that can be checked, and gives the same result for the same input. The language model is used to extract information from outlines (AIR3) and to explain suggestions in plain language. It never computes suggested dates or checks assessment weights, which are done by deterministic code.

- **The master course sheet is never sent to the language model** (AIR1, DR2). The sheet is confidential department data (DR1), and Copilot is an external service. Outlines word graduate attributes in different ways, so where the numbers or exact wording are missing, the model identifies the attributes from the learning outcome text (FR3). It needs only that public text and the public CEAB attribute definitions to do so, not the sheet. Comparing the identified attributes with the sheet (FR14) is then a lookup that ordinary code can do exactly. Attributes identified by the model are less certain than ones listed in the outline, so they are reviewed like every other extracted value (FR5, AIR6).

11.7 Numeric Values

The numeric values used in this document are symbolic constants defined in Appendix C (Table 9). Values marked with a section number come from the Undergraduate Course Management Policy [**?**]. Values marked "Goal" are the measurable success criteria agreed in the project's Problem Statement and Goals [**?**]. Values marked "Proposed" are the team's initial estimates. These will be reviewed with the CAS supervisors and adjusted based on Proof of Concept results.

12 Likely Changes

The following changes are expected during the life of the system. The design should hide each of them, so that a change affects as few components as possible. Table 7 lists the requirements each change affects.

LC1: **Language model and its hosting.** Copilot under McMaster's agreement [**?**] is preferred, and a locally hosted model is the fallback. Either choice may change, and models will be replaced as better ones are released.

LC2: **Course outline formats.** Instructors use different layouts. The university may also adopt a new outline template or syllabus platform, or the system may need to accept formats other than PDF.

LC3: **McMaster course management policy rules.** The Undergraduate Course Management Policy has been revised several times, most recently in 2025 [**?**]. Its required outline contents, weight limits, and restricted periods may change again.

LC4: **Accreditation criteria and the master course sheet.** The CEAB revises its criteria periodically [**?**]. The department may also change which graduate attributes each course is expected to cover.

LC5: **How graduate attributes are identified in an outline.** The system uses the attribute numbers or names in the outline where they are given, and the language model identifies them from the learning outcome text where they are not. This may change, for example if the department adopts an outline template that always lists the attribute numbers, or if the CEAB revises the attributes themselves [**?**].

LC6: **Definition of a workload cluster.** The values of *CLUSTER_WINDOW* and *CLUSTER_THRESHOLD* are initial estimates. Assessment types may also be weighted differently (for example, a midterm counting more than a quiz).

LC7: **Definition of a cohort.** How elective courses are grouped (A3) is provisional and will be clarified with the CAS department.

LC8: **Academic calendar dates.** Term dates, recesses, and examination periods change every year.

LC9: **Method for generating suggested dates.** The Development Plan [**?**] proposes a constraint solver, with a simpler scoring method as a fallback. The method may change as the Proof of Concept shows what works.

LC10: **How results reach instructors.** Email notification is on hold until it is confirmed with the CAS supervisors. Results may later be delivered by email or another channel.

LC11: **Export formats.** CSV is the only export format. Others, such as Excel or calendar files, may be requested.

LC12: **User authentication.** The system may move to McMaster's single sign-on if it is hosted on university infrastructure.

LC13: **Use of enrolment or timetable data.** Integration with Mosaic is a stretch goal. It would replace course-code cohorts (A1) with cohorts based on actual enrolment. Historical MSAF data, if made available, could also be used to identify weeks with high past assessment pressure.

LC14: **Scope of courses covered.** The project began as part of a larger initiative covering both undergraduate and graduate CAS programs. The system may later include graduate courses or other departments.

13 Unlikely Changes

The design may assume that the following will not change. Each item gives the reason it is considered stable.

ULC1: **Instructors keep final authority over assessment dates** (SR7, FR12). Under the Undergraduate Course Management Policy, instructors organize and present their courses, and changes to how a course is evaluated are made in consultation with the Department Chair [**?**]. CAS administrators do not have the authority to change an instructor's assessment dates, and a software system cannot be given authority that the people using it do not have.

ULC2: **Course outlines remain the main source of assessment information** (SR1). University policy requires every course to have an outline that lists its assessments, weights, and due dates [**?**]. The file format may change (LC2), but the outline itself will remain.

ULC3: **The first digit of a course number gives its year level** (A1). The Undergraduate Calendar defines course numbers this way for every McMaster course: the first digit gives the level of the course [**?**]. A change would affect every program and system at the university.

ULC4: **The two user roles: CAS administrator and instructor** (SR9, FR16). These roles reflect how outline review is divided in the department: administrators review outlines, and instructors own their courses. No other users will be given access.

ULC5: **Every assessment belongs to one course offering.** Academic records at McMaster are organized by course offering, and the course management policy is written at that level.

ULC6: **The system is used through a web browser** (NFR4, NFR9). Administrators and instructors use different devices and cannot be expected to install software. The Problem Statement and Goals [**?**] commits to a browser-based system with no additional hardware or software.

ULC7: **Level I courses are excluded.** First-year engineering is common to all programs and is not run by the CAS department. This would change only if the Faculty's first-year structure changed.

14 Appendices

14.1 Appendix C: Values of Auxiliary Constants

Table 9 gives the values of the symbolic constants used in this document. The Source column states where each value comes from: a section number (Sec.) refers to the Undergraduate Course Management Policy [**?**] ("App. A" to its Appendix A), "Goal" to the given goal in the project's Problem Statement and Goals document [**?**], and "Proposed" marks an initial estimate that will be confirmed with the CAS supervisors.

Constant	Value	Meaning	Source
Workload analysis			
CLUSTER_WINDOW	7 days	Length of the period checked for a workload cluster.	Proposed
CLUSTER_THRESHOLD	3	Number of assessments a cohort may have due within one window before a cluster is flagged.	Proposed
MIN_ALTERNATIVES	2	Minimum number of alternative dates offered for an assessment in a detected conflict.	Goal 4
University policy			
WEIGHT_TOTAL	100%	Required total of all assessment weights in an outline.	App. A
MAX_SINGLE_WEIGHT	75%	Largest weight one assessment may have without Chair or Associate Dean approval.	Sec. 13
BAN_CLASS_DAYS	5 days	Number of final days of classes included in the test and examination ban period.	Sec. 12.5
BAN_WEIGHT_LIMIT	10%	Largest weight a test or examination may have during the ban period.	Sec. 12.5
EARLY_FEEDBACK_L2	20%	Minimum weight of feedback given to Level II students before the last day to cancel a course.	Secs. 16–18
EARLY_FEEDBACK_L3	10%	Minimum weight of feedback given to Level III and higher students before the last day to cancel a course.	Secs. 16–18

Constant	Value	Meaning	Source
OUTLINE_LEAD_TIME	14 days	Recommended time before classes begin for an outline to be released.	Sec. 5
Success targets			
CALENDAR_COVERAGE	95%	Share of assessment dates in the test outlines that appear in the calendar.	Goal 1
EXTRACT_ACCURACY	95%	Share of assessment dates and weights extracted correctly (NFR1).	Proposed
FLAG_RECALL	95%	Share of known missing or incomplete fields that are flagged (NFR1).	Proposed
CONFLICT_RECALL	90%	Share of known scheduling conflicts identified.	Goal 3
SUGGEST_SUCCESS	90%	Share of conflict cases for which at least *MIN_ALTERNATIVES* feasible dates are offered.	Goal 4
COMPLIANCE_RECALL	95%	Share of known policy and CEAB compliance issues identified (NFR1).	Proposed
Nonfunctional targets			
TASK_COMPLETION	80%	Share of usability-test participants who complete each core task without assistance (NFR2).	Proposed
CALENDAR_LOAD_TIME	3 s	Longest time to display previously processed calendar data (NFR6).	Proposed
CALENDAR_LOAD_SHARE	95%	Share of calendar requests that must load within the time limit above.	Proposed
MAX_OUTLINE_PAGES	30	Longest outline, in pages, covered by the extraction time target.	Proposed
EXTRACT_TIME_LIMIT	60 s	Longest time to process one outline and extract its information (NFR6).	Proposed
EXTRACT_TIME_SHARE	90%	Share of extraction runs that must finish within the time limit above.	Proposed

Constant	Value	Meaning	Source
FRACTION	0.1	Largest share of the original development effort that any one likely change may require (NFR3).	Proposed

Table 9: Values of Auxiliary Constants

14.2 Appendix D: Generative AI Disclosure

14.3 AI Tools Used

The following AI tools were used:

- Grok (version 4.7)
- Claude (Anthropic), Claude Opus 5.5 model
- ChatGPT (OpenAI), GPT-4-turbo model
- GitHub Copilot (version 1.79.0)

14.4 How and Where Used

The tools were used in the following manner:

- Grok was used to fix formatting issues and address pull request comments in Sections 6, 7, 9.
- Claude was used to review and refine the document, suggesting improvements to structure and clarity, and assisting with grammar and style. It was used to primarily refine the Sections 10-13 and Appendix C section.
- ChatGPT and Copilot were used for formatting and grammar suggestions, and to assist with the writing of the Sections 1e, 2, and 8.

14.5 Verification of AI-Generated Content

Any output by LLM models was reviewed thoroughly and modified before its suggestions were added to the document. Course materials and deliverable requirements were manually reviewed to ensure that all technical assertions, requirements, and references were accurate and complete.

14.6 Appendix E: Reflection

1. **What went well while writing this deliverable?**

   We believe that the strongest part of the process was making useful progress by reviewing the document in small pull requests. Our team members carefully reviewed each section and highlighted duplicated descriptions, vague browser support, missing accessibility and deployment requirements, and missing or broken links. A team member also pointed out that the system-context figure, which was initially included without an explanation or legend, needed text explaining the relationships shown in the diagram. We added that explanation so the figure communicates system context rather than appearing to be included only to satisfy a documentation requirement.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   The main difficulty was converting a broad project vision into requirements that were specific enough to verify without pretending that uncertain design decisions had already been made. At first, we described the LLM, calendar, and recommendation features in terms of capabilities, but not in terms of failure behavior or evidence of success. Choosing numerical targets was also difficult. We addressed this by reviewing comparable document-processing and scheduling systems, consulting course requirements and project guidance, and discussing whether each target was realistic for our expected workload and resources.

   Another pain point was maintaining consistency across several documents. A safety roadmap initially used identifiers that were not the labels in the Safety and Security Requirements table, and some SRS sections repeated information instead of referencing it. We resolved this through our PR reviews.

3. **How many of your requirements were inspired by speaking to your client(s) or their proxies?**

   The core requirements came from the project brief and conversations with the CAS project supervisors (Matt & Connie): outline submission, extraction of assessment information, compliance checking, shared calendar views, workload-conflict detection, and instructor control over suggested dates.

4. **Which courses have helped, or will help, your team to be successful with the capstone project?**

   Our Software Requirements course helped us distinguish stakeholder needs from functional and non-functional requirements. The course also helped us realize that statements such as "balance workloads" are not testable until they are converted into a measurable recommendation outcome. Our Software Testing course helped us define representative datasets, invalid-input cases, and verification methods. Our Database course provided the foundation for storing course and assessment information and will

help us plan the structure of that information. Machine learning courses will be relevant to evaluating extraction quality, handling model uncertainty, and designing human review around LLM output.

5. **What knowledge and skills will the team collectively need to acquire?** Collectively, we need to build up our skills in a few areas to bring the different parts of the system together. This includes learning how to integrate an LLM, get consistent outputs from it, and handle issues like timeouts or unavailable services while keeping data private. We also need to figure out how to process course outline PDFs, including scanned documents, and turn the extracted information into assessment records that users can review. For scheduling, we need to learn how to use a constraint solver like CP-SAT to find assessment conflicts, account for restricted dates, and suggest better ways to spread out workloads without automatically changing dates. On the full-stack side, we need experience with database design, API development, authentication, and connecting the frontend to the different backend components. We also need to make sure the interface is accessible and test it against WCAG guidelines. Finally, we need to learn how to deploy and configure the system properly so that someone outside our team can set it up and maintain it.

6. **How will the team acquire this knowledge and master these skills?**

   The team will develop these skills through independent research, official documentation, tutorials, and hands-on experimentation. For unfamiliar technologies or tasks, we will conduct technical spikes (which we havea already started doing when evaluating llm models and testing our PDF text extraction libraries) to test different approaches, identify limitations, and determine the best solutions before implementation. We will document the findings of each spike on GitHub Discussions so that the whole team can refer back to them throughout development. We will also share knowledge through regular meetings and code reviews, and continue learning through implementation, troubleshooting, and testing as the project progresses.